COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 | Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	September 17, 2026	**DATE SUBMITTED:**	
NAME:	Santiago, Niña Denise M.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

HANDS-ON ACTIVITY NO. 9.1 **TREE ADT**

1. PROCEDURE OUTPUT

TASK 1 AND TASK 3:

Main.cpp

```cpp
#include <iostream>
#include "Tree.h"

int main() {
    // Task 1: Building tree nodes using your BST class style
    BinarySearchTree<char>* tree = new BinarySearchTree<char>('A');

    // Create child nodes
    BinarySearchTree<char>* nodeB = new BinarySearchTree<char>('B');
    BinarySearchTree<char>* nodeC = new BinarySearchTree<char>('C');
    BinarySearchTree<char>* nodeD = new BinarySearchTree<char>('D');
    BinarySearchTree<char>* nodeE = new BinarySearchTree<char>('E');
    BinarySearchTree<char>* nodeF = new BinarySearchTree<char>('F');
    BinarySearchTree<char>* nodeG = new BinarySearchTree<char>('G');
    BinarySearchTree<char>* nodeH = new BinarySearchTree<char>('H');
    BinarySearchTree<char>* nodeI = new BinarySearchTree<char>('I');
    BinarySearchTree<char>* nodeJ = new BinarySearchTree<char>('J');
    BinarySearchTree<char>* nodeK = new BinarySearchTree<char>('K');
    BinarySearchTree<char>* nodeL = new BinarySearchTree<char>('L');
    BinarySearchTree<char>* nodeM = new BinarySearchTree<char>('M');
    BinarySearchTree<char>* nodeN = new BinarySearchTree<char>('N');
    BinarySearchTree<char>* nodeP = new BinarySearchTree<char>('P');
    BinarySearchTree<char>* nodeQ = new BinarySearchTree<char>('Q');
```

```
    tree->addChild(nodeB);
    tree->addChild(nodeC);
    tree->addChild(nodeD);
    tree->addChild(nodeE);
    tree->addChild(nodeF);
    tree->addChild(nodeG);

    nodeD->addChild(nodeH);
    nodeE->addChild(nodeI);
    nodeE->addChild(nodeJ);
    nodeF->addChild(nodeK);
    nodeF->addChild(nodeL);
    nodeF->addChild(nodeM);
    nodeG->addChild(nodeN);
    nodeJ->addChild(nodeP);
    nodeJ->addChild(nodeQ);
```

```
    // Task 3.2: Output Traversals
    std::cout << "Pre Order Output: ";
    tree->preOrder();
    std::cout << std::endl;

    std::cout << "In Order Output: ";
    tree->inOrder();
    std::cout << std::endl;

    std::cout << "Post Order Output: ";
    tree->postOrder();
    std::cout << std::endl << std::endl;

    // Task 3.3: findData function test
    tree->findData("Pre-Order", 'P', tree);
    std::cout << std::endl;

    // Task 3.4: Add node 'O' as child of 'G' and search
    BinarySearchTree<char>* nodeO = new BinarySearchTree<char>('O');
    nodeG->addChild(nodeO);

    tree->findData("Pre-Order", 'O', tree);

    return 0;
}
```

Tree.h

```
#ifndef Tree_H
#define Tree_H

#include <iostream>
#include <string>

template <typename T>
class BinarySearchTree {
private:
    T key;
    BinarySearchTree<T>* firstChild;   // Replaces left pointer for general trees
    BinarySearchTree<T>* nextSibling;  // Replaces right pointer for general trees

    bool searchTree(BinarySearchTree<T>* node, T target) {
        if (node == nullptr) return false;
        if (node->key == target) return true;
        if (searchTree(node->firstChild, target)) return true;
        return searchTree(node->nextSibling, target);
    }

public:
    BinarySearchTree(T value);
    void addChild(BinarySearchTree<T>* child);
    BinarySearchTree<T>* findNode(T value);

    // Traversal algorithms
    void preOrder();
    void inOrder();
    void postOrder();

    // Search algorithm
    void findData(std::string choice, T target, BinarySearchTree<T>* root);
};
```

```cpp
// Constructor implementation
template <typename T>
BinarySearchTree<T>::BinarySearchTree(T value) {
    key = value;
    firstChild = nullptr;
    nextSibling = nullptr;
}

// Add a child node
template <typename T>
void BinarySearchTree<T>::addChild(BinarySearchTree<T>* child) {
    if (firstChild == nullptr) {
        firstChild = child;
    } else {
        BinarySearchTree<T>* temp = firstChild;
        while (temp->nextSibling != nullptr) {
            temp = temp->nextSibling;
        }
        temp->nextSibling = child;
    }
}

// Find node pointer by key
template <typename T>
BinarySearchTree<T>* BinarySearchTree<T>::findNode(T value) {
    if (key == value) return this;
    BinarySearchTree<T>* result = nullptr;
    if (firstChild != nullptr) result = firstChild->findNode(value);
    if (result == nullptr && nextSibling != nullptr) result = nextSibling->findNode(value);
    return result;
}
```

```cpp
// Pre-Order Traversal (Root -> Children)
template <typename T>
void BinarySearchTree<T>::preOrder() {
    std::cout << key << "    ";
    if (firstChild != nullptr) {
        firstChild->preOrder();
    }
    if (nextSibling != nullptr) {
        nextSibling->preOrder();
    }
}

// In-Order Traversal
template <typename T>
void BinarySearchTree<T>::inOrder() {
    if (firstChild != nullptr) {
        firstChild->inOrder();
    }
    std::cout << key << "    ";
    if (nextSibling != nullptr) {
        nextSibling->inOrder();
    }
}
```

```cpp
// Post-Order Traversal
template <typename T>
void BinarySearchTree<T>::postOrder() {
    if (firstChild != nullptr) {
        firstChild->postOrder();
    }
    std::cout << key << "    ";
    if (nextSibling != nullptr) {
        nextSibling->postOrder();
    }
}

// findData method
template <typename T>
void BinarySearchTree<T>::findData(std::string choice, T target, BinarySearchTree<T>* root) {
    std::cout << "Searching for '" << target << "' using [" << choice << "] traversal...\n";
    std::cout << "Traversal path: ";
    if (choice == "pre-order") {
        root->preOrder();
    } else if (choice == "in-order") {
        root->inOrder();
    } else if (choice == "post-order") {
        root->postOrder();
    }
    std::cout << std::endl;

    if (searchTree(root, target)) {
        std::cout << "\"" << target << " was found!\"\n";
    }
}

#endif
```

TASK 2

NODE	HEIGHT	DEPTH
A	3	0
B	O	1
C	0	1
D	1	1
E	2	1
F	1	1
G	1	1
H	O	2
I	0	2
J	1	2
K	0	2
L	0	2
M	0	2
N	0	2
P	0	3
Q	0	3

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Code + Console Screenshot + Tree Diagram

```
#ifndef BinarySearchTree_H
#define BinarySearchTree_H

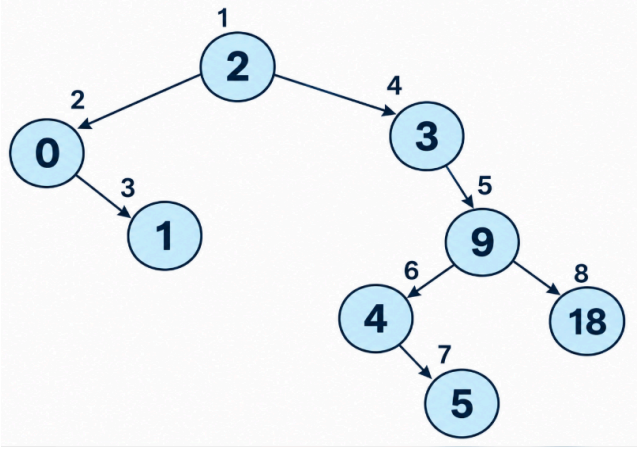

template <typename T>
class BinarySearchTree{
    private:
        T key;
        BinarySearchTree<T>* left;
        BinarySearchTree<T>* right;

    public:
        // constructor
        BinarySearchTree(T value);
        // insertion algorithm:
        void insert(T value);

        // traversal
        void preOrder();
        void inOrder();
        void postOrder();
};
```

```
    std::cout<<"Pre Order Output: ";
    tree->preOrder();
    std::cout<<std::endl;
```

```
Pre Order Output: 2    0    1    3    9    4    5    18
```



Observation	The pre-order traversal code visits the root first, followed by its children. In the output, the values are displayed as 2, 0, 1, 3, 9, 4, 5, and 18. The code first prints the current node before recursively checking the left and right nodes. This is why 2, the root node, appears first. The output follows the structure of the tree from the top and then moves through the child nodes. The result shows that the pre-order algorithm correctly visits the nodes in Root → Left → Right order.

Table 1. Creation of Pre-Order Traversal

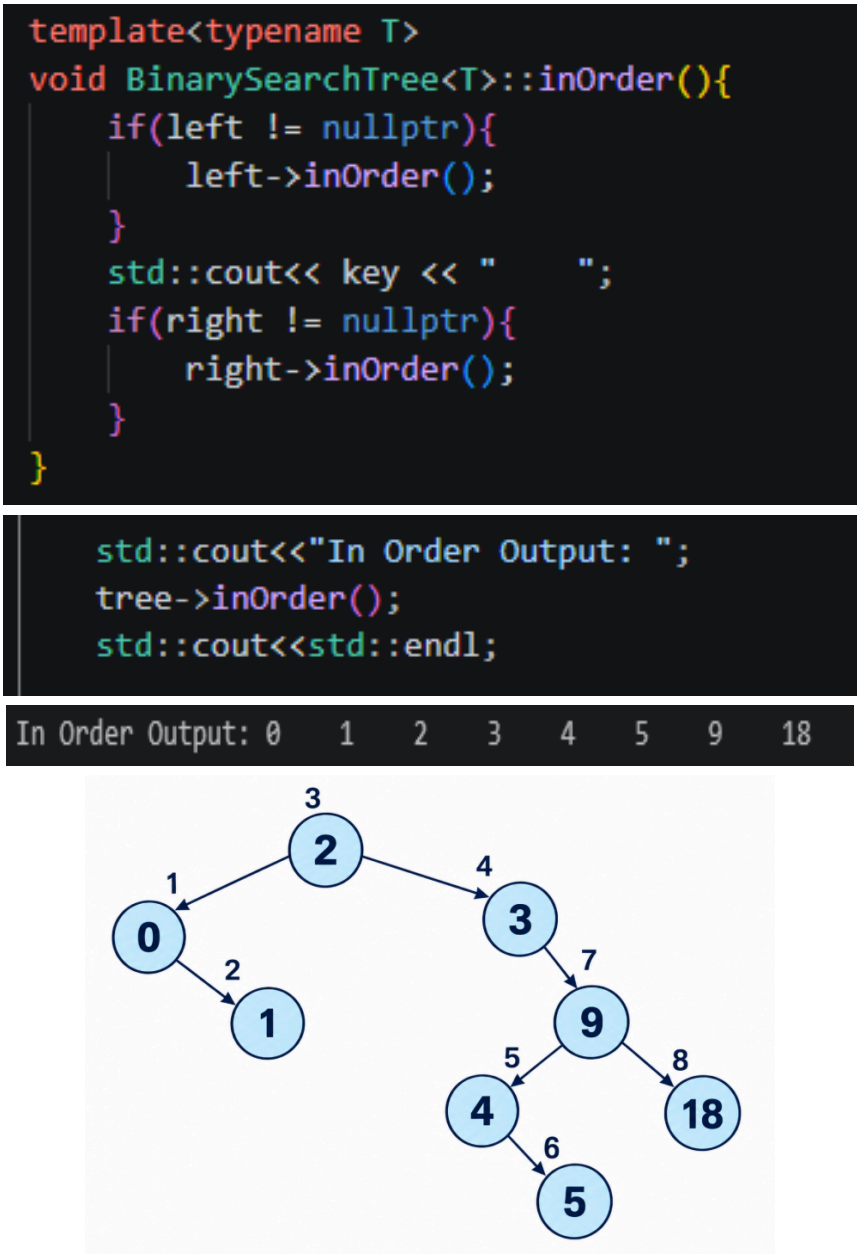
Code + Console Screenshot + Tree Diagram	
Observation	The in-order traversal visits the left child first, then the current node, and finally the right child. The output is 0, 1, 2, 3, 4, 5, 9, and 18. In the code, the program first checks the left node before printing the current key, then checks the right node. Because of this order, the root value 2 is printed after the values on its left side. The output shows that the algorithm correctly follows the Left → Root → Right order.

Table 2. Creation of In-Order Traversal

Code + Console Screenshot + Tree Diagram

```
template<typename T>
void BinarySearchTree<T>::postOrder(){
    if(left != nullptr){
        left->postOrder();
    }
    if(right != nullptr){
        right->postOrder();
    }
    std::cout<< key << "    ";

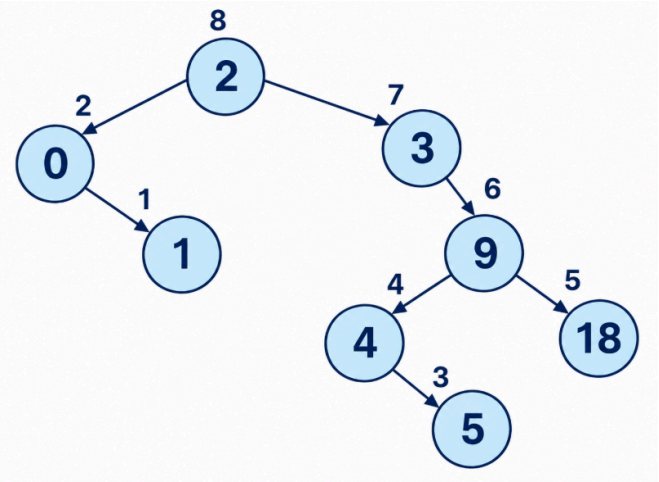
}

#endif
```

```
    std::cout<<"Post Order Output: ";
    tree->postOrder();
    std::cout<<std::endl;

    return 0;
}
```

```
Post Order Output: 1    0    5    4    18    9    3    2
```



Observation

The post-order traversal visits the left child and right child before the current node. The

	console output is 1, 0, 5, 4, 18, 9, 3, and 2. The code first calls the post-order function for the left and right nodes and only prints the current key afterward. Because of this, the root node 2 is printed last. The output shows that the traversal correctly follows the Left → Right → Root order. This demonstrates how changing the position of the std::cout statement changes the order in which the tree nodes are displayed.

Table 3. Creation of Post-Order Traversal

CONCLUSION / LEARNINGS / ANALYSIS:
This laboratory activity helped me understand how a Binary Search Tree organizes data and how the position of each node affects its height and depth. I learned how pre-order, in-order, and post-order traversals visit nodes in different sequences. By comparing the tree diagram with the console output, I was able to see how the recursive functions work and verify that the traversal results were correct. Overall, the activity improved my understanding of tree structures, recursion, and traversal algorithms in C++.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Grammarly and Microsoft Copilot

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Grammar Checker & Code Assistance

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: The AI helped me understand the code as I was absent last Laboratory. It guided me how to create and understand the algorithm of Binary Search Tree.
I also used Grammarly to check my grammatical errors. The AI did not replace any original thinking, analysis, authorship, or completion of the required work.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.



Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.